

Architecture et Programmation Web

Séance 6 : ORM avec SQLAlchemy

Nicolas Burny

27 mars 2026

ECAM

Le Problème du SQL Brut

Pourquoi aller au-delà du SQL ?

- **Mélange des langages** : Mélange de SQL et Python dans la partie serveur.
- **Syntaxes variables** : SQL n'est pas 100% identique entre les différents moteurs SQL (SQLite, PostgreSQL, MySQL, etc.).
- **Maintenance** : Difficile de suivre les changements de modèle dans tout le projet.
- **Erreurs à l'exécution** : Une faute de frappe dans une requête SQL ne se voit qu'à l'exécution.

Object Relational Mapping

- Convertit les **tables** en **classes**.
- Convertit les **lignes** de table en **objets** Python.
- Permet de manipuler la base de données avec le langage utilisé pour la partie serveur.

SQLAlchemy et Flask

- **SQLAlchemy** : un des ORM les plus utilisés dans l'écosystème Python.
- **Flask-SQLAlchemy** : Une extension qui facilite l'intégration dans Flask (gestion des sessions, configuration).

Installation :

```
pip install flask-sqlalchemy
```

Configuration de base

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
# Chemin vers le fichier SQLite
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///test.db'
db = SQLAlchemy(app)
```

Évolution de l'API

Flask-SQLAlchemy $\geq$ 3.x utilise SQLAlchemy 2.x. L'ancienne syntaxe `Model.query` fonctionne toujours mais n'est plus recommandée.

Legacy (1.x)	Moderne (2.x)
<code>Feature.query.all()</code>	<code>db.session.execute(db.select(Feature))</code>
<code>Feature.query.get(1)</code>	<code>db.session.get(Feature, 1)</code>
<code>.filter_by(x=1)</code>	<code>.where(Feature.x == 1)</code>

- Dans ce cours, nous utilisons la syntaxe **legacy** pour sa lisibilité.
- La documentation officielle utilise la syntaxe **moderne**.

Définition des Modèles

Structure d'un modèle

```
class Feature(db.Model):  
    __tablename__ = 'features'  # Nom de la table en DB  
  
    id = db.Column(db.Integer, primary_key=True)  
    title = db.Column(db.String(100), nullable=False)  
    description = db.Column(db.Text)  
    is_active = db.Column(db.Boolean, default=True)  
  
    def __repr__(self):  
        return f'<Feature {self.id}: {self.title}>'
```

- `__tablename__` : Nom explicite de la table (sinon dérivé du nom de classe en minuscules).
- `__repr__` : Représentation lisible pour le débogage en console.

Types de Colonnes principaux

Type SQLAlchemy	Équivalent SQL
<code>db.Integer</code>	INTEGER
<code>db.String(taille)</code>	VARCHAR
<code>db.Text</code>	TEXT (Long)
<code>db.Boolean</code>	BOOLEAN
<code>db.DateTime</code>	DATETIME

Avant de pouvoir stocker des données, il faut que SQLAlchemy crée les tables physiques en fonction des modèles Python.

```
with app.app_context():  
    db.create_all()
```

- `db.create_all()` crée uniquement les tables qui **n'existent pas encore**.
- Si vous modifiez un modèle existant, il faut supprimer le fichier `.db` ou utiliser des migrations (pas vues dans ce cours).

Manipulation des Données (CRUD)

Create (Ajouter)

1. Instancier l'objet

```
new_f = Feature(title="Tri", description="Trier par date")
```

2. L'ajouter à la session

```
db.session.add(new_f)
```

3. Valider la transaction

```
db.session.commit()
```

Requêtes : Read

Récupérer toutes les lignes

```
all_items = Feature.query.all()
```

Par identifiant unique (PK)

```
f = Feature.query.get(1)
```

```
f = Feature.query.get_or_404(1)  # Erreur 404 si introuvable
```

Filtrer par valeur exacte

```
urgent = Feature.query.filter_by(priority='Haute').all()
```

Filtrer avec opérateurs

```
f = Feature.query.filter(Feature.id > 10).first()
```

Requêtes : Update

1. On récupère l'objet

```
f = Feature.query.get(1)
```

2. On change ses attributs

```
f.title = "Nouveau Titre"
```

3. On enregistre (SQLAlchemy détecte le changement)

```
db.session.commit()
```

```
f = Feature.query.get(1)

# Préparer la suppression
db.session.delete(f)

# Valider
db.session.commit()
```

Requêtes : Tri et Pagination

```
from sqlalchemy import desc

# Trier par un champ (croissant par défaut)
items = Feature.query.order_by(Feature.title).all()

# Trier par ordre décroissant
items = Feature.query.order_by(desc(Feature.id)).all()

# Pagination : ignorer les 10 premiers, prendre les 5 suivants
items = Feature.query.offset(10).limit(5).all()
```

Requêtes : Filtres Avancés

Recherche textuelle (LIKE SQL)

```
bugs = Feature.query.filter(Feature.title.contains('bug')).all()
```

Recherche dans une liste (IN)

```
urgent = Feature.query.filter(Feature.priority.in_(['Haute', 'Critique']))  
        .all()
```

Vérifier les valeurs nulles (IS NULL)

```
orphans = Feature.query.filter(Feature.user_id == None).all()
```

Requêtes : Combinaisons Logiques

```
from sqlalchemy import and_, or_, not_

# Plusieurs conditions (AND par défaut dans filter)
f = Feature.query.filter(Feature.id > 10, Feature.priority == 'Basse')
    .all()

# Condition OU (OR)
f = Feature.query.filter(or_(
    Feature.priority == 'Haute',
    Feature.user_id == 1
)).all()

# Négation (NOT)
f = Feature.query.filter(not_(Feature.done)).all()
```

Les Relations

Pourquoi définir des relations ?

Les données sont rarement isolées. Les relations permettent de lier les entités entre elles de manière logique et structurée.

- **Côté SQL** : On utilise des **Foreign Keys** (clés étrangères) pour pointer d'une table vers une autre.
- **Côté ORM** : On transforme ces liens en **propriétés d'objets** pour naviguer intuitivement dans les données sans écrire de JOIN manuels.
- **Types de relations courants** :
 - **1 :N (One-to-Many)** : Un utilisateur possède plusieurs articles.
 - **N :N (Many-to-Many)** : Un étudiant est inscrit à plusieurs cours (et vice-versa).
 - **1 :1 (One-to-One)** : Un utilisateur possède un unique profil de préférences.

Relation One-to-Many (1 :N)

```
class User(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    features = db.relationship('Feature', backref='author')  
  
class Feature(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'))
```

1. **Côté "Many" (Feature)** : On place la ForeignKey. Elle pointe vers l'ID du parent.
2. **Côté "One" (User)** : On utilise `relationship()`.
 - `backref='author'` : Ajoute virtuellement un attribut `.author` sur chaque Feature.
 - `cascade='all, delete-orphan'` : Supprime les Features liées quand on supprime le User.
 - `lazy='select'` (défaut) : Charge les relations à la demande. Utiliser `lazy='joined'` pour charger immédiatement via un JOIN.

```
# Créer avec relation
user1 = User(username="John")
feat1 = Feature(title="Test", author=user1) # On passe l'OBJET

# Naviguer
print(feat1.author.username) # Sortie: John

# Historique
print(user1.features) # Sortie: [<Feature Test>]
```

Relation Many-to-Many (N :M)

```
# Table de jonction (pas un modèle, juste une table)
user_groups = db.Table('user_groups',
    db.Column('user_id', db.Integer,
               db.ForeignKey('user.id')),
    db.Column('group_id', db.Integer,
               db.ForeignKey('group.id'))
)

class Group(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(50))
    members = db.relationship('User', secondary=user_groups,
                              backref='groups')
```

- secondary= indique la table de jonction.
- backref='groups' ajoute user.groups automatiquement.

Sessions et Transactions

- **La Session** : C'est l'espace de travail local (le "staging area"). Elle garde une trace de tous les objets chargés ou modifiés durant une requête.
- **La Transaction** : C'est une unité de travail atomique au niveau de la base de données (principe du "Tout ou Rien").
- **Le lien** : Une session peut encapsuler plusieurs transactions successives. Elle accumule les changements et ne les rend permanents qu'au `commit()`.

Exemple : Transactions Successives

Transaction 1 : Création de l'utilisateur

```
user = User(username="Alice")
```

```
db.session.add(user)
```

```
db.session.commit()
```

Transaction 2 : Ajout d'une entité liée

L'objet 'user' reste utilisable après le premier commit

```
feat = Feature(title="Refonte UI", author=user)
```

```
db.session.add(feat)
```

```
db.session.commit()
```

Chaque commit valide une unité de travail distincte

- `db.session.add()` : Place l'objet dans la session pour qu'il soit suivi et prêt à être persisté.
- `db.session.commit()` : Enregistre les modifications dans la base de données.
- `db.session.rollback()` : Annule les modifications et vide la session.

Cohérence

La base de données n'est pas modifiée tant que `commit()` n'est pas appelé !

```
try:
    new_feature = Feature(title="Export CSV")
    db.session.add(new_feature)
    db.session.commit()
except Exception:
    db.session.rollback() # Annule tout en cas d'erreur
    raise # Remonte l'exception
```

- Toujours appeler `rollback()` en cas d'échec du `commit()`.
- Sinon la session reste dans un état incohérent pour les requêtes suivantes.

- **Abstraction** : On manipule des objets Python plutôt que des chaînes SQL.
- **Productivité** : Les relations complexes (One-to-Many) sont gérées automatiquement.
- **Sécurité** : Protection native contre les injections SQL.
- **Portabilité** : Le code reste identique même si l'on change de moteur de base de données (SQLite vers PostgreSQL).

1. **Définition** : Création des classes héritant de `db.Model`.
2. **Instanciation** : Création d'un objet Python (`u = User(...)`).
3. **Suivi** : Ajout à la session (`db.session.add(u)`).
4. **Persistence** : Enregistrement définitif (`db.session.commit()`).

- **Imports circulaires** : Attention lors de la définition des relations entre fichiers.
- **Performance** : Éviter les boucles qui font des milliers de petits `commit()`.
- **Migrations** : Toute modification du modèle nécessite une mise à jour du schéma (via Flask-Migrate) (pas vu dans ce cours.)

- **Table SQL** → **Classe Python** héritant de `db.Model`
- **Ligne** → **Instance** de la classe
- **Foreign Key** → `db.ForeignKey` + `db.relationship`
- **CRUD** : `add/commit` (C), `query` (R), `modifier + commit` (U), `delete` (D)
- **Session** : Espace de travail local, validé par `commit()`

Questions ?